Exp No:20Experiment to understand K-means clustering algorithm for a given data set.

Description: Understand the K-means clustering algorithm for the dataset given.

Code:

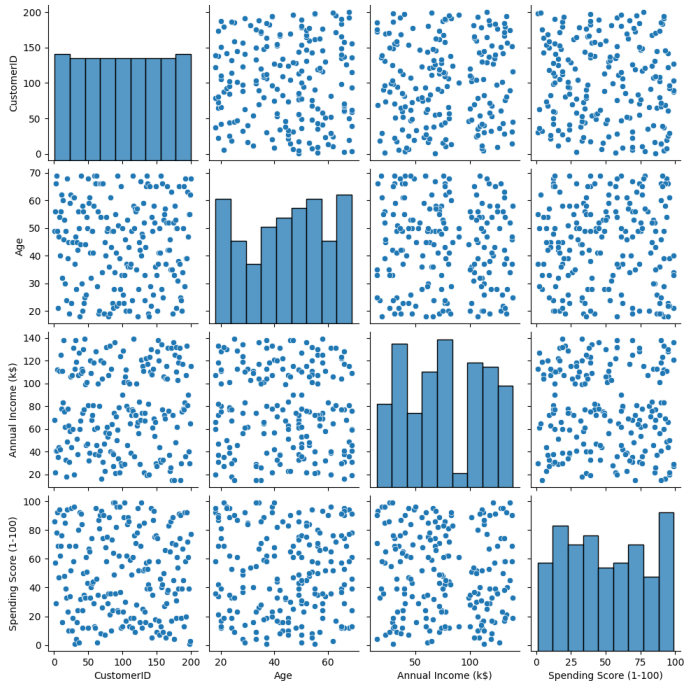
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('/content/Mall_Customers.csv')
df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #  Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0  CustomerID              200 non-null    int64
 1  Gender                  200 non-null    object
 2  Age                     200 non-null    int64
 3  Annual Income (k$)      200 non-null    int64
 4  Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	49	68	86
1	2	Female	56	22	57

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
2	3	Male	66	41	29
3	4	Male	69	41	78
4	5	Male	49	112	92

```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x7841ea0a0ad0>
```




```
features=df.iloc[:,[3,4]].values
```

```
from sklearn.cluster import KMeans
```

```
model=KMeans(n_clusters=5)
```

```
model.fit(features)
```

```
KMeans(n_clusters=5)
```

```
Final=df.iloc[:,[3,4]]
```

```
Final['label']=model.predict(features)
```

```
Final.head()
```

```
/tmp/ipykernel_2901/470183701.py:2: SettingWithCopyWarning:
```

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

Final['label']=model.predict(features)

	Annual Income (k$)	Spending Score (1-100)	label
0	68	86	4
1	22	57	0
2	41	29	0
3	41	78	4
4	112	92	1

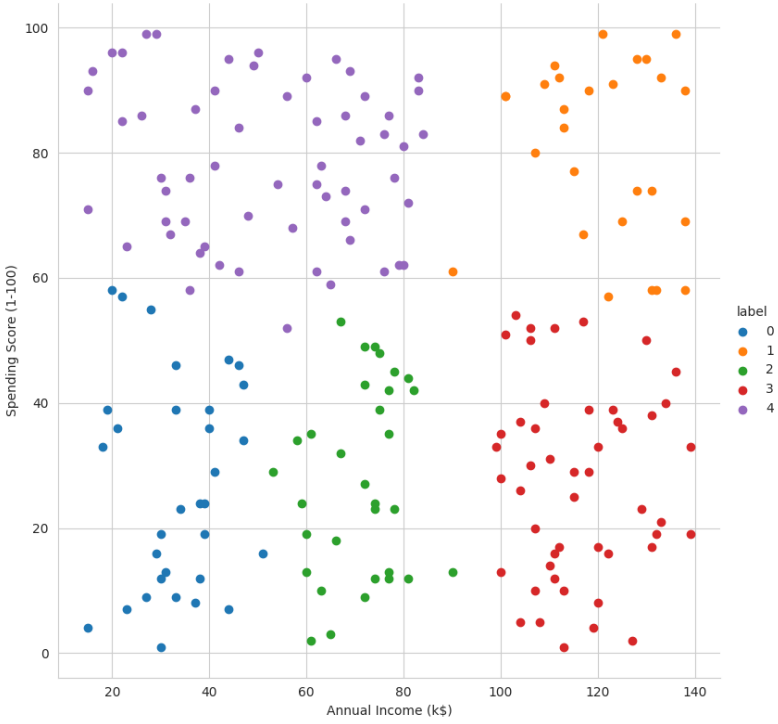
```
sns.set_style("whitegrid")

sns.FacetGrid(Final,hue="label",height=8) \

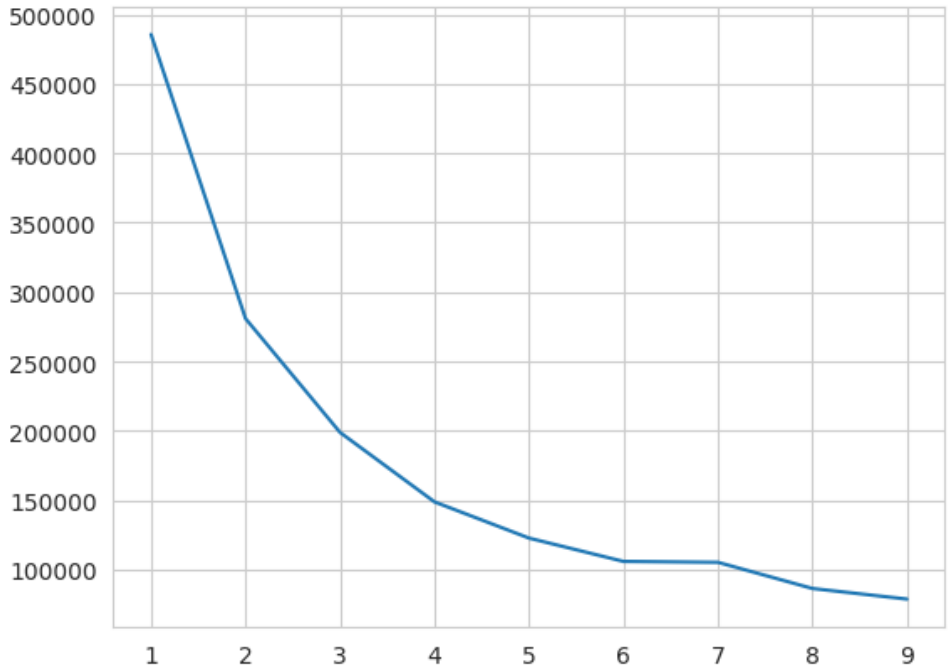
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \

.add_legend();

plt.show()
```



```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
 model=KMeans(n_clusters=i)
 model.fit(features_el)
 wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
[<matplotlib.lines.Line2D at 0x7841dd2a0e10>]
```



Result:

Understood the K-means clustering algorithm for the given dataset.